[ML] 2주차 예습과제_권지원

Ch2. 사이킷런으로 시작하는 머신러닝

1. 사이킷런 소개와 특징

- 머신러닝을 위한 매우 다양한 알고리즘과 개발을 위한 편리한 프레임워크와 API 제공
- 오랜 기간 실전 환경에서 검증되었으며, 매우 많은 환경에서 사용되는 라이브러리

```
pip install -U scikit-learn
```

```
import sklearn
print(sklearn.version)
```

2. 첫 번째 머신러닝 만들어보기: 붓꽃 품종 예측하기

- 붓꽃 데이터 세트로 붓꽃의 품종을 분류 (Classification)
- 꽃잎의 길이와 너비, 꽃받침의 길이와 너비 피처(Feature) 기반으로 꽃의 품종을 예측
- 분류 (Classification) - 지도학습 (Supervised Learning)
- 지도 학습: 학습을 위한 다양한 Feature와 분류 결정값인 Label 데이터 로 모델을 학습한 뒤, 별도의 테스트 데이터 세트에서 미지의 Label을 예측함 → 명확한 정답이 주어진 데이터를 학습 후, 미지의 정답을 예측.
- 사이킷런 패키지 내의 모듈명: sklearn으로 시작
- 붓꽃 데이터 로딩: load_iris(), ML 알고리즘 : 의사결정 트리 (Decision Tree) 알고리즘 → DecisionTreeClassifier
- train_test_split(): 데이터 세트를 학습 데이터와 테스트 데이터로 분리 → 학습된 데이터로 학습된 모델이 얼마나 뛰어난 성능을 가지는지 평가

---

1. 붓꽃 데이터 세트 예측 프로세스

Feature

- sepal length
- sepal width
- petal length
- petal width

Label

- 0 : Setosa 품종
- 1 : Versicolor 품종
- 2 : Virginica 품종

---

2. 데이터 세트 분리

학습 데이터와 테스트 데이터를 **test_size 파라미터 입력값의 비율로 쉽게 분할**

```
X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label,
test_size=0.2)
```

파라미터

- iris_data : 피처 데이터 세트
- iris_label : Label 데이터 세트
- test_size=0.2 : 전체 데이터 세트 중 테스트 데이터 세트의 비율
- random_state : 난수 발생 값
  - random_state 미지정 시 수행할 때마다 다른 학습/테스트 데이터 생성

데이터 구성

- X_train : 학습용 피처 데이터 세트
- X_test : 테스트용 피처 데이터 세트
- y_train : 학습용 레이블 데이터 세트
- y_test : 테스트용 레이블 데이터 세트

---

3. DecisionTreeClassifier 모델 생성 및 학습

```
# DecisionTreeClassifier 객체 생성
dt_clf = DecisionTreeClassifier(random_state=11)
```

```
# 학습 수행
dt_clf.fit(X_train, y_train)
```

---

4. 예측 수행

```
# 테스트 데이터로 예측 수행
pred = dt_clf.predict(X_test)
```

- predict 메서드에 피처 데이터 세트를 입력해서 호출
- 학습된 모델 기반에서 테스트 데이터 세트에 대한 **예측값 반환**

---

5. 모델 평가

의사결정트리 기반 **DecisionTreeClassifier의 예측 성능 평가**

- `accuracy_score()` 이용
- 정확도 측정

---

6. 붓꽃 데이터 세트 예측 프로세스 정리

1. **데이터 세트 분리**
   - 데이터를 학습 데이터와 테스트 데이터로 분리
2. **모델 학습**
   - 학습 데이터를 기반으로 ML 알고리즘을 적용하여 모델 학습
3. **예측 수행**
   - 학습된 ML 모델을 이용해 테스트 데이터 분류 예측
4. **평가**
   - 예측 결과와 실제 결과를 비교하여 모델 성능 평가

---

7. 사이킷런의 기반 프레임워크

사이킷런은 ML 모델 학습을 위해 **fit()** 제공

학습된 모델 예측을 위해 **predict()** 제공

Estimator 구조

구분	설명
Classifier	분류 알고리즘을 구현한 클래스
Regressor	회귀 알고리즘을 구현한 클래스

→ **Estimator 클래스 = Classifier + Regressor**

---

8. 사이킷런 주요 모듈

구분	모듈	설명
예제 데이터	sklearn.datasets	사이킷런에 내장된 예제 데이터 세트
피처 처리	sklearn.preprocessing	데이터 전처리 기능
피처 처리	sklearn.feature_selection	중요 피처 선택 기능
피처 처리	sklearn.feature_extraction	텍스트/이미지 데이터 벡터화
차원 축소	sklearn.decomposition	차원 축소 알고리즘
데이터 분리/검증	sklearn.model_selection	데이터 분리, 교차검증, GridSearch
평가	sklearn.metrics	분류·회귀·클러스터링 성능 평가
ML 알고리즘	sklearn.ensemble	랜덤포레스트, AdaBoost, Gradient Boost
ML 알고리즘	sklearn.linear_model	선형회귀, Ridge, Lasso, 로지스틱 회귀
ML 알고리즘	sklearn.naive_bayes	나이브 베이즈
ML 알고리즘	sklearn.neighbors	K-NN

구분	모듈	설명
ML 알고리즘	sklearn.svm	SVM
ML 알고리즘	sklearn.tree	의사결정트리
ML 알고리즘	sklearn.cluster	K-means, 계층형, DBSCAN

---

9. 사이킷런 내장 데이터 세트

내장 데이터 세트는 **일반적으로 딕셔너리 형태**

fetch 계열 명령

- 패키지에 처음부터 저장 X
- 인터넷에서 다운로드 후
- `scikit_learn_data` 디렉터리에 저장

---

load_iris() 반환 데이터 구조

반환 객체 : **sklearn.utils.Bunch 클래스**

Key	설명
data	피처 데이터 (ndarray)
target	분류 : Label / 회귀 : 숫자 결과값
target_names	개별 label 이름
feature_names	피처 이름
DESCR	데이터 세트 설명

피처 데이터 추출

```
dataset.data
```

- 데이터 세트가 딕셔너리 형태 → `.data` 이용

---

10. Model Selection 모듈

train_test_split()

원본 데이터 세트에서 **학습/테스트 데이터 세트 분리**

주의

- 학습 데이터로 평가하면 **정확도 100% 가능 → 의미 없음**
- 반드시 **별도 테스트 데이터로 평가**

---

11. 교차 검증 (Cross Validation)

Overfitting (과적합)

모델이 **학습 데이터에만 과도하게 최적화된 상태**

→ 새로운 데이터에서 성능 급락

교차 검증

여러 개의 학습/검증 세트를 이용하여 **반복 평가**

대부분의 ML 모델 평가 과정

1차 평가 → **교차 검증**

최종 평가 → **테스트 데이터**

---

12. K-Fold 교차 검증

과정

1. 데이터 세트를 K등분
2. 1개 폴드 → 검증 데이터
3. 나머지 → 학습 데이터
4. 학습 후 검증 평가
5. K번 반복
6. 평균 성능 계산

사이킷런 클래스

- `KFold`
- `StratifiedKFold`

---

13. Stratified K-Fold

레이블 데이터 분포를 고려하여 **동일한 비율로 분할**

특징

- 레이블 분포 유지
- split()에 **피처 + 레이블 모두 필요**

결과 예시

→ 평균 검증 정확도 약 **96.67%**

주의

- 왜곡된 레이블 데이터에서는 반드시 사용
- **회귀 문제에서는 지원 X**

---

14. cross_val_score()

교차 검증을 **간단하게 수행하는 API**

```
cross_val_score(estimator, X, y=None, scoring=None, cv=None, n_jobs=1)
```

파라미터

- estimator : Classifier 또는 Regressor
- X : 피처 데이터
- y : 레이블 데이터
- scoring : 평가 지표
- cv : 교차 검증 폴드 수

특징

- 내부에서 자동 수행
  - fit()
  - predict()
  - evaluation

관련 API

`cross_validate()`

→ 여러 평가 지표 반환 가능

---

15. GridSearchCV

교차 검증 + 하이퍼파라미터 튜닝

하이퍼파라미터

머신러닝 알고리즘의 구성 요소

→ 조정 시 성능 개선 가능

```
grid_parameters = {
 'max_depth':[1,2,3],
 'min_samples_split':[2,3]
}
```

주요 파라미터

- estimator
- param_grid
- scoring
- cv
- refit

결과 정보

- `rank_test_score` : 성능 순위
- `mean_test_score` : 평균 평가 점수

---

16. 데이터 전처리

결손값 처리

결손값 (NaN, NULL) 허용 X

대처 방법

- 평균값 대체
- 피처 삭제

텍스트 피처

- 벡터화
- 삭제

---

17. 데이터 인코딩

1) 레이블 인코딩

카테고리 값을 **숫자로 변환**

예

TV → 1

냉장고 → 2

전자레인지 → 3

컴퓨터 → 4

선풍기 → 5

믹서 → 6

사용 클래스

`LabelEncoder`

```
fit()
transform()
```

주의

숫자 **대소 관계가 생기므로 ML 알고리즘에 문제 발생 가능**

---

2) 원-핫 인코딩

레이블 인코딩 문제 해결

특징

- 새로운 피처 생성
- 해당 값만 1
- 나머지 0

사용 클래스

`OneHotEncoder`

특징

- 입력은 **2차원 데이터 필요**
- 결과는 **희소 행렬**

```
toarray()
```

→ 밀집 행렬 변환

Pandas API

`get_dummies()`

문자열 카테고리 → 바로 원핫 인코딩

---

18. 피처 스케일링

서로 다른 변수 범위를 **일정 수준으로 맞추는 작업**

종류

- 표준화
- 정규화

표준화

피처 평균 = 0

분산 = 1

가우시안 정규 분포 형태

StandardScaler

```
fit()
transform()
```

정규화

피처 크기 통일

Normalizer

MinMaxScaler

데이터를 **0~1 범위로 변환**

음수

→ -1 ~ 1 범위

데이터가 **가우시안 분포가 아닐 때 사용**

---

19. 스케일링 적용 시 주의점

학습 데이터와 테스트 데이터 스케일링 시 주의

1. **학습 데이터로 fit() 수행**
2. **테스트 데이터는 transform()만 수행**

이유

테스트 데이터에 다시 fit() 수행하면

→ **학습 데이터 기준과 달라짐**

---

20. 정리

- 사이킷런은 다양한 머신러닝 기능 제공 패키지
- 머신러닝 애플리케이션 과정
  1. 전처리
  2. 데이터 분리
  3. ML 알고리즘 적용
  4. 모델 학습
- 모델 학습 → **학습 데이터**
- 모델 평가 → **테스트 데이터**

교차 검증

- KFold
- StratifiedKFold
- cross_val_score

하이퍼파라미터 튜닝

- GridSearchCV

→ **파이썬 기반 머신러닝 필수 패키지**

Ch3. 평가

---

0. 분류 모델 성능 평가 지표

주요 평가 지표

- 정확도 (Accuracy)
- 오차 행렬 (Confusion Matrix)
- 정밀도 (Precision)
- 재현율 (Recall)
- F1 스코어
- ROC-AUC

분류 유형

- **이진 분류 (Binary Classification)**
- **멀티 분류 (Multi-class Classification)**

---

01. 정확도 (Accuracy)

정확도 정의

정확도 = **예측 결과가 동일한 데이터 건수 / 전체 예측 데이터 건수**

→ 직관적으로 모델 예측 성능을 나타내는 평가 지표

Dummy Classifier 예시

`BaseEstimator` 클래스를 상속하여

학습 없이 **단순한 Classifier 생성**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# 원본 데이터 재로딩
# 데이터 가공
# 학습 데이터 / 테스트 데이터 분할

from google.colab import drive
drive.mount('/content/drive/')

titanic_df = pd.read_csv("/content/drive/MyDrive/titanic_train.csv")

y_titanic_df = titanic_df['Survived']
X_titanic_df = titanic_df.drop('Survived', axis=1)
```

```
X_titanic_df = transform_features(X_titanic_df)

X_train, X_test, y_train, y_test = train_test_split(
    X_titanic_df,
    y_titanic_df,
    test_size
)
```

결과 예시

```
Dummy Classifier의 정확도: 0.7877
```

정확도의 맹점

데이터 분포가 **균일하지 않은 경우 높은 수치가 나타날 수 있음**

예시

- **불균형 데이터 세트**
- Dummy Classifier 생성 → 예측 및 평가

예시 데이터

- **MNIST 데이터 세트 (0~9 숫자 이미지 픽셀 정보)**

---

02. 오차 행렬 (Confusion Matrix)

정의

오차 행렬

→ 분류 모델이 예측하면서 **어떤 유형의 오류가 발생하는지** 보여줌

즉,

- 이진 분류의 예측 오류 수
- 어떤 유형의 예측 오류가 발생했는지

구성

실제 / 예측	Negative(0)	Positive(1)
Negative(0)	TN	FP
Positive(1)	FN	TP

의미

- **TN (True Negative)**
  - 0으로 예측
  - 실제도 0

- **FP (False Positive)**
  - 1로 예측
  - 실제는 0
- **FN (False Negative)**
  - 0으로 예측
  - 실제는 1
- **TP (True Positive)**
  - 1로 예측
  - 실제도 1
- ⇒ 불균형한 이진 데이터 분류 세트 → Negative로 예측 정확도가 높아지는 경향 발생

API

```
confusion_matrix()
```

정확도 수식

정확도

[

Accuracy = (TN + TP) / (TN + FP + FN + TP)

]

특징

불균형 이진 데이터 분류 세트에서는

→ **Negative 예측 정확도가 높아지는 경향**

---

03. 정밀도 (Precision)와 재현율 (Recall)

정밀도 (Precision)

정밀도 = **TP / (FP + TP)**

→ Positive로 **예측한 대상 중 실제로 Positive인 비율**

재현율 (Recall, 민감도)

재현율 = **TP / (FN + TP)**

→ 실제 Positive 중 **모델이 맞게 예측한 비율**

중요성

분류 모델의 **업무 특성에 따라 중요한 지표가 달라짐**

계산 API

- 정밀도 → `precision_score()`
- 재현율 → `recall_score()`

정밀도 / 재현율 트레이드오프

정밀도와 재현율은 **상호 보완적인 지표**

한쪽을 높이면 다른 쪽은 낮아지기 쉬움

---

predict_proba()

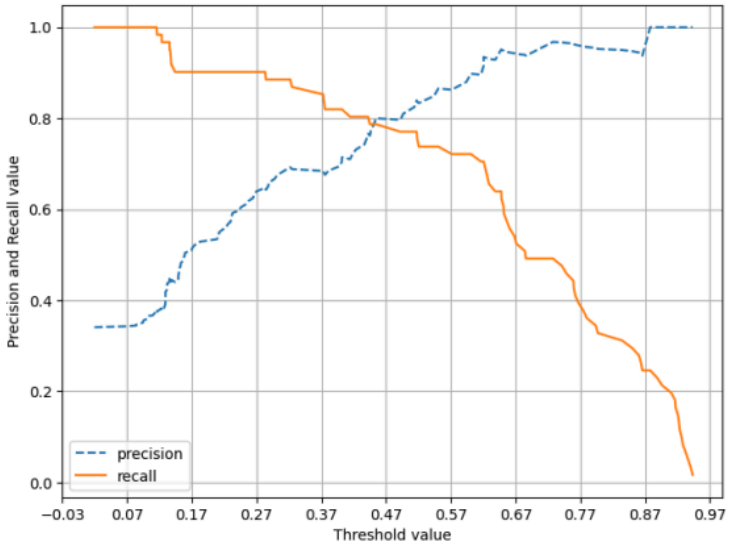
테스트 피처 데이터를 입력하면

각 클래스 예측 확률 반환

```
from sklearn.preprocessing import Binarizer

# Binarizer의 threshold 설정값 (분류 결정 임곗값)
custom_threshold = 0.5

# positive 클래스 확률만 추출
pred_proba_1 = pred_proba[:,1].reshape(-1,1)

binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_1)

custom_predict = binarizer.transform(pred_proba_1)

get_clf_eval(y_test, custom_predict)
```

Threshold 조정 효과

- **임곗값 낮추면**
  - 재현율 ↑
  - 정밀도 ↓



---

precision_recall_curve()

정밀도 / 재현율 관계 계산

입력 파라미터

- `y_true`
- `probas_pred`

반환값

- 정밀도
- 재현율

→ 정밀도-재현율 곡선 시각화

정밀도 / 재현율의 맹점

정밀도 100% 방법 → **확실한 경우만 Positive**

재현율 100% 방법 → **모든 데이터를 Positive로 예측**

---

04. F1 스코어

정의

정밀도와 재현율을 결합한 평가 지표

정밀도와 재현율이 **균형을 이룰 때 높은 값**

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

API

```
f1_score()
```

예시 코드

```
from sklearn.metrics import f1_score

f1 = f1_score(y_test, pred)
print('F1 스코어: {0:.4f}'.format(f1))
```

평가 함수 예시

```
def get_clf_eval(y_test, pred):

    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)

    # F1 스코어
    f1 = f1_score(y_test, pred)

    print('오차 행렬')
    print(confusion)

    print('정확도:{0:.4f}, 정밀도:{1:.4f}, 재현율:{2:.4f}, F1:{3:.4f}'.format(
        accuracy, precision, recall, f1))
```

Threshold별 평가

```
thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]

pred_proba = lr_clf.predict_proba(X_test)

get_eval_by_threshold(
    y_test,
    pred_proba[:,1].reshape(-1,1),
    thresholds
)
```

05. ROC 곡선과 AUC

ROC 곡선

ROC (Receiver Operating Characteristic Curve)

→ **FPR 변화에 따른 TPR 변화 곡선**

정의

- **TPR (True Positive Rate)**

  = 재현율

- **FPR (False Positive Rate)**

TNR (특이성)

TNR = **TN / (FP + TN)**

FPR 계산

$$FPR = FP/(FP + TN)$$

$$FPR = 1 - TNR$$

ROC 해석

- 가운데 직선에 가까울수록 → 성능 낮음
- 멀어질수록 → 성능 좋음

API

```
roc_curve()
```

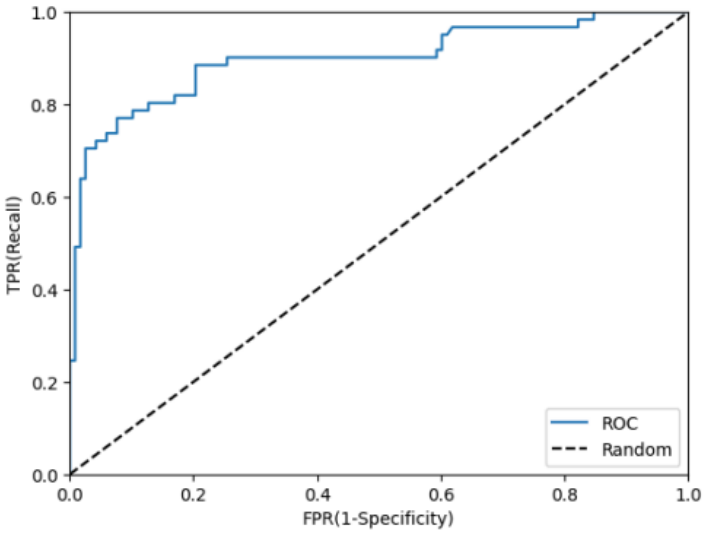
입력 파라미터

- `y_true`
- `y_score`

반환값

- fpr
- tpr
- thresholds

ROC 시각화



FPR 변화에 따른 **TPR 변화 곡선**

AUC

정의

ROC 곡선 아래 면적

→ **1에 가까울수록 성능이 좋음**

특징

ROC-AUC는 **예측 확률값 기반 계산**

```
get_clf_eval(y_test, pred=None, pred_proba=None)
```

형태로 함수 변경 가능

---

07. 정리

분류 성능 평가 지표

- 정확도
- 오차 행렬
- 정밀도
- 재현율
- F1 스코어
- ROC-AUC

오차 행렬

실제 클래스 & 예측 클래스

- TN
- FP
- FN
- TP

정밀도 / 재현율

Positive 데이터 예측 성능 중심 지표

→ **Threshold 조정으로 값 변경 가능**

F1 스코어

정밀도 + 재현율 결합 지표

→ 균형일 때 높은 값

ROC-AUC

이진 분류 성능 평가 지표

→ AUC는 ROC 곡선 아래 면적!